

DESARROLLO DE SOFTWARE

Resumen completo para parcial

Ingeniería en Sistemas - UTN FRVM

Arquitectura web, REST, capas, SOLID, code smells, testing, Git y microservicios

Objetivo del apunte

Entender los conceptos para poder justificar decisiones de diseño en casos prácticos, como los del modelo de parcial: identificar problemas, proponer una solución y defenderla con vocabulario técnico.

Fuentes base: PDFs de cátedra Semana 02, Semana 03 y Semana 05. Complemento: documentación MDN, Git/GitHub, AWS, Martin Fowler, Jest, RFC JWT y NestJS.

Índice de estudio

Unidad	Tema	Qué te llevás
1	Cómo contestar el parcial	Estructura de respuesta, palabras clave y errores comunes.
2	Arquitectura cliente-servidor, web y HTTP	Componentes, request/response, métodos, códigos y HTTPS.
3	API y API REST	Recursos, URI, JSON, endpoints, paginación, jerarquías, idempotencia.
4	Arquitectura por capas	Dominio, repositorios, services, controladores, validaciones y errores.
5	Cualidades de diseño	Cohesión, acoplamiento, simplicidad, robustez, flexibilidad.
6	SOLID y code smells	Principios con ejemplos y cómo usarlos para justificar refactorizaciones.
7	Testing	Condiciones/casos de prueba, unitarios, mocks y Jest.
8	Git, GitFlow y pull requests	Ramas, trabajo remoto, revisiones y estrategia antes del merge.
9	Monolito, monolito modular y microservicios	Cuándo elegir cada enfoque y cómo migrar con bajo riesgo.
10	Seguridad web: JWT, middleware y guards	Resumen útil para preguntas de autenticación/autorización.
11	Respuestas modelo	Resolución guiada de casos similares al parcial.
12	Machete final	Checklist de conceptos para repasar antes de entrar.

1. Cómo contestar bien un parcial de Desarrollo de Software

En esta materia no alcanza con nombrar una tecnología: hay que justificar una decisión de diseño. La respuesta ideal suele tener tres partes: diagnóstico, propuesta y justificación.

Paso	Qué escribir	Ejemplo
1. Diagnóstico	Identificar el problema de diseño o arquitectura.	La clase GestorUsuarios concentra registrar usuarios, validar contraseñas, enviar correos y auditar accesos; por lo tanto tiene baja cohesión y alta responsabilidad.
2. Principio o criterio	Nombrar el concepto que se viola o se busca mejorar.	Viola SRP, aumenta acoplamiento y dificulta testing.
3. Propuesta	Separar responsabilidades, introducir capas, interfaces, repositorios, servicios o estrategia de Git según corresponda.	Separaría UserService, AuthService, EmailService, AuditService y UserRepository.
4. Justificación	Explicar qué gana el diseño:	Así cada clase tiene una razón de

	mantenibilidad, flexibilidad, robustez, testeabilidad, escalabilidad, colaboración.	cambio, se puede mockear el mail y cambiar la base de datos sin tocar dominio.
--	---	--

Frase comodín para justificar

'Esta decisión reduce el acoplamiento, aumenta la cohesión y permite evolucionar el sistema con menor impacto, porque cada componente queda responsable de una parte clara del problema.'

Errores comunes en respuestas

- Decir solo 'uso microservicios' sin explicar por qué el sistema los necesita ni sus costos.
- Diseñar rutas REST con verbos: /crearUsuario, /eliminarUsuario. En REST el verbo HTTP expresa la acción y la URL expresa el recurso.
- Poner toda la lógica en el controlador. El controlador traduce la request y delega; el negocio va en dominio/services.
- Confundir actor con clase responsable: que un Alumno use el sistema no significa que la clase Alumno deba tener verCalificaciones().
- Hablar de testing como 'probar que funciona' sin definir condición de prueba, caso de prueba, resultado esperado y uso de mocks.

Basado en modelo de parcial provisto y PDFs de cátedra.

2. Arquitectura cliente-servidor, web y HTTP

Cliente-servidor

Una arquitectura cliente-servidor divide el sistema en dos grandes roles: el cliente consume servicios y el servidor los provee a través de una interfaz. En una aplicación web, el cliente suele ser un navegador o una app frontend, y el servidor expone datos o funcionalidades mediante HTTP/HTTPS.

Tipo	Dónde está la lógica	Ejemplo	Riesgo principal
Cliente activo - Servidor pasivo	Gran parte de la lógica está en el cliente; el servidor persiste datos.	Aplicación de escritorio que consulta una base remota.	Cliente pesado y difícil de actualizar.
Cliente pasivo - Servidor activo	El servidor concentra la lógica; el cliente presenta datos.	Web tradicional o API consumida por frontend liviano.	Servidor como cuello de botella o punto único de falla si no se replica.
Cliente activo - Servidor activo	Lógica distribuida entre frontend y backend.	SPA con validaciones y reglas de presentación + backend con reglas de negocio.	Duplicación de reglas o inconsistencias si no se separa bien.

Cátedra DDS Semana 02: cliente-servidor y arquitectura web.

HTTP en una idea

HTTP es el protocolo de aplicación usado para intercambiar datos en la web. Funciona con un ciclo de solicitud y respuesta: el cliente envía una request; el servidor la procesa y devuelve una response con datos o con un código de estado.

- Es cliente/servidor: alguien pide, alguien responde.
- Es stateless: cada request debe traer la información necesaria para ser interpretada; el servidor no debería depender de recordar la request anterior.
- Usa métodos o verbos para expresar intención: GET, POST, PUT, PATCH, DELETE.
- Usa códigos de estado para indicar el resultado: 2xx éxito, 4xx error del cliente, 5xx error del servidor.
- HTTPS es HTTP sobre TLS: agrega cifrado, integridad y autenticación del servidor.

Cátedra DDS Semana 02 + MDN HTTP Methods/Status.

Métodos HTTP principales

Método	Uso correcto	Ejemplo REST	Idempotencia práctica
GET	Consultar un recurso sin modificarlo.	GET /users/10	Sí: repetirlo no debería cambiar el estado.
POST	Crear un recurso o disparar un proceso.	POST /users	No necesariamente: repetirlo puede crear dos usuarios.
PUT	Reemplazar completamente un recurso o crearlo con ID conocido.	PUT /users/10	Sí: enviar el mismo cuerpo varias veces deja el recurso igual.
PATCH	Modificar parcialmente un recurso.	PATCH /users/10	Depende de cómo se diseñe.
DELETE	Eliminar un recurso.	DELETE /users/10	Sí en efecto final: tras varias llamadas el recurso sigue eliminado.

Idempotencia

Una operación es idempotente cuando repetir la misma solicitud varias veces no produce cambios adicionales en el estado final del sistema. Es importante para reintentos, fallos de red y APIs robustas.

Códigos HTTP que más se usan en parcial

Código	Significado	Cuándo usarlo
200 OK	La operación salió bien y hay respuesta.	GET /users/10 o PUT/PATCH que devuelve el recurso actualizado.
201 Created	Se creó un recurso.	POST /users luego de registrar un usuario.
204 No Content	La operación salió bien pero no hay cuerpo de respuesta.	DELETE /users/10.
400 Bad Request	La request está mal formada o faltan datos.	Email inválido, campos obligatorios ausentes.
401 Unauthorized	No está autenticado o el token es inválido.	Falta JWT o está vencido.
403 Forbidden	Está autenticado pero no tiene permisos.	Usuario común intenta acceder a ruta admin.
404 Not Found	El recurso no existe.	GET /users/999 si no existe.
409 Conflict	Hay conflicto con el estado actual.	Registrar email ya existente.
500 Internal Server Error	Falla inesperada del servidor.	Excepción no controlada, caída de dependencia.

Mini ejemplo de request/response

```
GET /users/10 HTTP/1.1
Host: api.ejemplo.com
Authorization: Bearer <token>

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "nombre": "Ana",
  "email": "ana@mail.com"
}
```

3. API y API REST

API

Una API es una interfaz que permite que un sistema use capacidades de otro sin conocer su implementación interna. Puede ser una API de lenguaje, de sistema operativo, de base de datos o una API web. En el parcial, cuando se habla de API normalmente se espera pensar en endpoints, recursos, HTTP y JSON.

REST

REST no es una librería ni un framework: es un estilo de diseño para comunicar cliente y servidor. En una API REST se modelan recursos, se identifican con URI y se manipulan con métodos HTTP. El formato de intercambio más habitual es JSON.

- La URL representa recursos: /users, /users/10, /courses/3/students.
- La acción no va como verbo en la URL: no usar /crearUsuario ni /borrarUsuario.
- El método HTTP expresa la operación: GET consulta, POST crea, PUT/PATCH actualiza, DELETE elimina.
- Los códigos HTTP comunican el resultado.
- La request debe llevar la información necesaria: path params, query params, body, headers, token, etc.

Cátedra DDS Semana 02: API REST, recursos web, JSON/XML, rutas y códigos.

Cómo diseñar endpoints REST

Caso de uso	Endpoint	Body	Respuesta esperada
Listar usuarios	GET /users?page=1&pageSize=20	No	200 OK + lista paginada.
Obtener usuario	GET /users/{id}	No	200 OK + usuario, o 404 si no existe.
Crear usuario	POST /users	nombre, email, password	201 Created + usuario creado sin password.
Actualizar usuario	PUT /users/{id}	representación completa	200 OK + usuario actualizado.
Actualizar parcialmente	PATCH /users/{id}	solo campos a cambiar	200 OK + usuario actualizado.
Eliminar usuario	DELETE /users/{id}	No	204 No Content.
Cambiar contraseña	POST /users/{id}/password-changes	actual, nueva	201 Created o 204, según diseño.
Asignar rol	POST /users/{id}/roles	roleId	201 Created o 200 OK.

Regla para procesos

Cuando una acción no encaja como ABM simple, conviene modelarla como recurso de proceso. Ejemplo: en vez de POST /validarLicitacion/200, usar POST /validations con { licitacionId: 200 } y luego GET /validations/{id} para consultar el resultado.

URL, path params y query string

Parte	Qué representa	Ejemplo
Dominio	Dónde está el servidor.	https://api.ejemplo.com
Base/path	Recurso o jerarquía de recursos.	/users/10/orders
Path param	Identificador obligatorio dentro del path.	/users/{id}
Query string	Filtros, paginación, ordenamiento o búsqueda.	?page=2&pageSize=20&status=active

Paginación

Cuando una lista puede crecer mucho, no conviene devolver todos los datos. Se usan parámetros de paginación y una respuesta que indique página, tamaño y total.

```
GET /products?page=2&pageSize=50&sort=name
```

```
{
  "data": [ { "id": 1, "name": "Mouse" } ],
  "page": 2,
  "pageSize": 50,
  "total": 500
}
```

JSON: cómo debería verse una respuesta limpia

```
{
  "id": 5,
  "name": "Victoria",
  "email": "victoria@mail.com",
  "roles": ["student", "admin"],
  "createdAt": "2026-06-10T18:20:00Z"
}
```

- No devolver password ni hash de password.
- No mezclar detalles internos de base de datos si no son parte del contrato de la API.
- Mantener nombres consistentes: usar siempre camelCase o snake_case, no mezclar.
- Usar mensajes de error claros pero sin revelar datos sensibles del servidor.

4. Arquitectura por capas y orquestación de casos de uso

Idea central

No hay que asignar automáticamente responsabilidades a las clases que representan actores. Que el actor 'Alumno' vea calificaciones no significa que la entidad Alumno deba tener un método verCalificaciones(). La lógica del caso de uso debe orquestarse en una capa de aplicación/servicio, mientras que las reglas propias del negocio viven en el dominio.

Cátedra DDS Semana 03: orquestación de casos de uso y separación de dominio, servicios y controladores.

Frase clave

El foco no está en quién usa el sistema, sino en qué operación de negocio se ejecuta y qué objetos colaboran para resolverla.

Flujo típico de una API por capas

```
Cliente HTTP
-> Controller: recibe request, valida forma básica, traduce a DTO
-> Service / Caso de uso: orquesta pasos y reglas de flujo
    -> Dominio: entidades y objetos de valor aplican reglas de negocio
    -> Repository: obtiene o guarda datos
<- Service devuelve resultado
<- Controller arma response + código HTTP
```

Responsabilidad de cada capa

Capa	Responsabilidad	Qué NO debería hacer
Dominio	Reglas de negocio puras, entidades, objetos de valor,	No depender de Express, Nest, SQL, HTTP ni permisos externos.

	invariantes.	
Repository	Encapsular acceso a persistencia y traducir entre dominio y almacenamiento.	No decidir reglas de negocio complejas.
Service / Caso de uso	Orquestrar una operación: buscar entidades, invocar dominio, persistir, coordinar dependencias.	No transformarse en God Service con todas las reglas y detalles de infraestructura.
Controller	Recibir request, validar estructura, llamar al service y devolver response/código.	No contener lógica de negocio real.
Middleware / Guard / Interceptor	Preprocesamiento, autenticación/autorización, cross-cutting concerns.	No resolver casos de uso completos.

Ejemplo: registrar calificación

Paso	Quién lo hace	Por qué
Validar que nota esté entre 0 y 10	Dominio: Calificacion	Es una regla propia de la calificación.
Buscar alumno y materia	Service usando repositorios	El caso de uso necesita datos persistidos.
Agregar calificación al alumno	Entidad Alumno	Es comportamiento del dominio.
Guardar cambios	Repository	Persistencia queda aislada.
Devolver 201 Created	Controller	Es una decisión HTTP de presentación/API.

```
// Controller: traduce HTTP y delega
POST /students/123/grades
body = { subjectId: 4, value: 8 }

// Service: orquesta
registrarCalificacion({ studentId, subjectId, value }) {
  const student = studentRepo.findById(studentId);
  const subject = subjectRepo.findById(subjectId);
  const grade = new Grade(subject, value); // regla de dominio
  student.addGrade(grade);
  studentRepo.save(student);
  return grade;
}
```

Validaciones por nivel

Nivel	Ejemplo	Objetivo
UI / Frontend	Mostrar 'email inválido' antes de enviar.	Mejorar experiencia de usuario.
Controller	Verificar que exista el campo email y que sea string.	Rechazar requests mal formadas.
Service	Verificar que no exista otro usuario con ese email.	Regla de caso de uso / consistencia.
Dominio	PasswordPolicy valida longitud y complejidad.	Mantener invariantes del modelo.
Persistencia	UNIQUE(email), NOT NULL, foreign keys.	Última defensa de integridad.

Errores y excepciones

Un buen manejo de errores evita mezclar responsabilidades. El dominio o servicio puede lanzar errores significativos; el controlador o filtro global los traduce a HTTP.

```
class ValidationError extends Error {}
class NotFoundError extends Error {}

function errorHandler(err, req, res, next) {
  if (err instanceof ValidationError) return res.status(400).json({ error: err.message });
  if (err instanceof NotFoundError) return res.status(404).json({ error: err.message });
  return res.status(500).json({ error: 'Error interno del servidor' });
}
```

- 400: el cliente mandó datos inválidos.
- 404: el recurso pedido no existe.
- 409: conflicto con datos existentes, por ejemplo email duplicado.
- 500: error no esperado; no exponer stack trace al cliente.

5. Cualidades de diseño

Las cualidades de diseño son criterios para comparar alternativas. En un parcial, sirven para justificar por qué una solución es mejor que otra. No son absolutas: muchas veces se gana en una cualidad y se paga con otra.

Cátedra DDS Semana 05: cualidades de diseño, SOLID, code smells y testing.

Cualidad	Qué significa	Cómo se mejora	Ejemplo de parcial
Cohesión	Qué tan enfocadas están las responsabilidades de un componente.	Separar clases por responsabilidad.	GestorUsuarios no debería registrar, validar, auditar y enviar emails todo junto.
Acoplamiento	Grado de dependencia entre componentes.	Usar interfaces, inyección de dependencias, eventos o contratos claros.	UserService depende de EmailSender, no de GmailSMTP concreto.
Simplicidad	Evitar complejidad innecesaria.	Aplicar KISS y YAGNI.	No crear microservicios para un sistema académico simple de tres personas.
Robustez	Responder bien ante errores, datos inválidos o fallas externas.	Validaciones, manejo de excepciones, logs, estados consistentes.	Si falla el email, no dejar usuario a medio crear sin control.
Flexibilidad	Adaptarse a cambios del dominio.	Diseñar módulos extensibles y mantenibles.	Agregar nuevo canal de notificación sin tocar UserService.
Extensibilidad	Agregar características con bajo impacto.	OCP, estrategias, plugins.	Agregar autenticación por Google como nueva estrategia.
Mantenibilidad	Modificar lo existente con menor esfuerzo y riesgo.	Código claro, bajo acoplamiento, tests.	Cambiar política de contraseña en una clase PasswordPolicy.

KISS y YAGNI

- KISS: Keep It Simple. No agregar arquitectura, patrones o capas que no aporten al problema actual.
- YAGNI: You Aren't Gonna Need It. No implementar funcionalidades 'por si algún día'.
- Aplicación típica: si el equipo es chico y el dominio todavía no está estable, conviene comenzar con un monolito modular bien separado antes que microservicios.

Cómo usar cualidades en una justificación

'Elijo una arquitectura por capas porque aumenta la mantenibilidad y testeabilidad. El costo es agregar más clases, pero se justifica porque separa reglas de negocio, acceso a datos y transporte HTTP.'

6. Principios SOLID

SOLID es un conjunto de principios de diseño orientado a objetos que ayuda a evitar código difícil de mantener. En el parcial conviene nombrar el principio, explicar la violación y mostrar cómo se rediseña.

Letra	Principio	Idea en una frase	Pregunta que me hago
S	Single Responsibility	Una clase debe tener una única razón para cambiar.	¿Esta clase hace demasiadas cosas?
O	Open/Closed	Abierto a extensión, cerrado a modificación.	¿Puedo agregar un comportamiento sin tocar lo existente?
L	Liskov Substitution	Un subtipo debe poder reemplazar al tipo base sin romper el programa.	¿La subclase cumple la promesa de la clase padre?
I	Interface Segregation	Interfaces pequeñas y específicas son mejores que una gigante.	¿Estoy obligando a implementar métodos que no se usan?
D	Dependency Inversion	Depender de abstracciones, no de implementaciones concretas.	¿Mi clase conoce demasiado detalle técnico de otra?

Cátedra DDS Semana 05: principios SOLID y ejemplos.

S - Single Responsibility Principle

Una clase tiene baja cohesión cuando concentra responsabilidades que cambian por motivos distintos. Si cambia la política de contraseña, el envío de correo o la auditoría, no debería modificarse la misma clase.

```
// Mal: muchas razones para cambiar
class GestorUsuarios {
  registrarUsuario() {}
  validarPassword() {}
  enviarEmailBienvenida() {}
  auditarAcceso() {}
}

// Mejor: responsabilidades separadas
class UserService {}
class PasswordPolicy {}
class EmailService {}
class AuditService {}
```

O - Open/Closed Principle

El código debería permitir agregar variantes sin modificar la clase central. Se suele lograr con polimorfismo, interfaces, estrategias o inyección de dependencias.

```
interface NotificationSender {
    send(to, message): Promise<void>;
}

class EmailSender implements NotificationSender {}
class SmsSender implements NotificationSender {}
class WhatsAppSender implements NotificationSender {}

class NotificationService {
    constructor(private sender: NotificationSender) {}
}
// Para agregar WhatsApp no modifico NotificationService; agrego otra implementación.
```

L - Liskov Substitution Principle

Si una clase hija hereda de una clase padre, debe poder usarse donde se esperaba el padre sin generar comportamientos sorpresivos. Una subclase que lanza 'no soportado' en métodos esenciales suele violar LSP.

```
// Mal: una impresora vieja hereda scan() pero no puede escanear
class MultifunctionMachine { print() {}; scan() {}; fax() {}; }
class OldPrinter extends MultifunctionMachine {
    scan() { throw new Error('No soportado'); }
}

// Mejor: separar capacidades
interface Printer { print(); }
interface Scanner { scan(); }
```

I - Interface Segregation Principle

No obligar a los clientes a depender de métodos que no usan. En vez de una interfaz enorme, conviene tener interfaces específicas por necesidad.

D - Dependency Inversion Principle

Los módulos de alto nivel no deberían depender de detalles de bajo nivel. Por ejemplo, un caso de uso no debería crear directamente MySQLDatabase o GmailClient; debería depender de interfaces.

```
// Mal
class UserService {
    constructor() { this.repo = new MySQLUserRepository(); }
}

// Mejor
class UserService {
    constructor(userRepository, emailSender) {
        this.userRepository = userRepository;
        this.emailSender = emailSender;
    }
}
```

Relación entre capas y SOLID

Las capas ayudan especialmente a SRP y DIP: controller no hace negocio, service no sabe de HTTP, dominio no sabe de base de datos, y las dependencias concretas se inyectan desde afuera.

7. Code smells

Un code smell no significa automáticamente 'código incorrecto'. Es una señal de alerta: tal vez el diseño funciona, pero puede ser difícil de cambiar, testear o entender. En el parcial se usa para diagnosticar.

Smell	Cómo se ve	Qué problema genera	Refactor posible
Código duplicado	Misma lógica repetida en varios lugares.	Cambiar una regla obliga a recordar todos los lugares.	Extraer función, clase o regla de dominio.
Método largo	Un método hace validaciones, consultas, reglas, logs y respuesta.	Difícil de entender y testear.	Extraer métodos o separar service/dominio.
Parámetros largos	Funciones con muchos parámetros sueltos.	Firma frágil y difícil de usar.	Crear DTO, objeto de comando o value object.
God Class	Clase central que sabe y hace demasiado.	Viola SRP, baja cohesión y alto acoplamiento.	Dividir en clases especializadas.
Herencia rechazada	Subclase que no usa o rompe métodos heredados.	Jerarquía forzada; posible violación LSP.	Composición o interfaces más chicas.
Método fuera de lugar	Método que usa más datos de otra clase que de la propia.	Responsabilidad mal ubicada.	Move Method hacia la clase correcta.
Intermediario innecesario	Clase que solo redirige llamadas sin aportar valor.	Complejidad accidental.	Eliminar o fusionar si no agrega abstracción útil.

Cómo escribir diagnóstico con smells

La clase GestorUsuarios presenta el smell God Class porque concentra registro, validación, correo y auditoría. Esto viola SRP, reduce cohesión y aumenta el acoplamiento con infraestructura externa. La refactorización propuesta es separar servicios especializados e introducir interfaces para EmailSender y AuditLogger.

8. Testing

Qué es testing

Testing es una verificación dinámica: se ejecuta el sistema o un componente bajo condiciones específicas y se observa si el resultado coincide con lo esperado. Una prueba es valiosa si puede encontrar fallas importantes de forma rápida y barata.

Cátedra DDS Semana 05: definición de testing, condiciones y casos de prueba.

Concepto	Definición	Ejemplo
Condición de prueba	Situación que quiero verificar.	No se debe registrar un usuario con email inválido.
Caso de prueba	Datos concretos para ejecutar esa condición.	email='ana', password='abc12345' => resultado esperado: ValidationError.
Resultado esperado	Comportamiento correcto.	HTTP 400 + mensaje de error.
Falla	Diferencia entre resultado real y esperado.	El sistema crea el usuario con email inválido.

Testing unitario

Un test unitario verifica una unidad pequeña de código aislada: una función, entidad, servicio o caso de uso. Para aislarla, se reemplazan dependencias externas por mocks o stubs.

- Rápido: no debería depender de red, base de datos real ni servicios externos.
- Específico: si falla, debería señalar una causa concreta.
- Repetible: debería dar el mismo resultado siempre.
- Aislado: las dependencias se simulan con mocks cuando corresponde.

Mocks

Un mock reemplaza una dependencia real para controlar su comportamiento y observar cómo se la llamó. Es útil para testear un service sin enviar emails reales ni consultar una base real.

Jest Docs: mock functions y manual mocks.

```
test('registra usuario y envía email', async () => {
  const repo = { save: jest.fn() };
  const email = { send: jest.fn() };
  const service = new UserService(repo, email);

  await service.register({ email: 'ana@mail.com', password: 'abc12345' });

  expect(repo.save).toHaveBeenCalled();
  expect(email.send).toHaveBeenCalledWith('ana@mail.com', expect.any(String));
});
```

Qué testear en cada capa

Capa	Qué testear	Cómo
Dominio	Reglas puras: nota válida, contraseña válida, estado permitido.	Tests unitarios sin mocks o con mínimos objetos.
Service	Flujo del caso de uso: busca, valida, crea, guarda, notifica.	Mocks de repositorios y servicios externos.
Controller	Mapeo request-response, códigos HTTP, validaciones básicas.	Tests de integración o controller con service mockeado.
Repository	Consultas y mapeo de persistencia.	Tests de integración con base de prueba.
API completa	Contrato externo: endpoints y respuestas.	Tests end-to-end.

Respuesta modelo sobre testing

'Probaría primero reglas de dominio con unit tests, luego services con repositorios mockeados y finalmente endpoints críticos con tests de integración/e2e. Esto permite detectar fallas rápido y evita depender de servicios externos en cada ejecución.'

9. Git, ramas, GitFlow y pull requests

En equipos remotos, Git no solo sirve para guardar versiones: ordena la colaboración. Permite trabajar en ramas separadas, revisar cambios antes de integrarlos y volver atrás si algo rompe el sistema.

Git Book: branching/merging. GitHub Docs: pull requests y reviews.

Conceptos básicos

Concepto	Qué significa	Para qué sirve
Commit	Foto lógica de cambios con mensaje.	Registrar avance pequeño y entendible.
Branch	Línea paralela de trabajo.	Desarrollar features o fixes sin romper main.
Merge	Integrar cambios de una rama a otra.	Unir una feature terminada a develop/main.
Pull request	Propuesta de cambios para revisar y mergear.	Discutir, revisar diff, ejecutar checks y aprobar.
Conflict	Dos ramas modifican la misma zona de forma incompatible.	Obliga a decidir manualmente qué queda.

GitFlow

GitFlow es una estrategia de ramas más formal. Suele ser útil cuando hay releases planificados, varias features en paralelo y necesidad de separar desarrollo, estabilización y producción.

Rama	Uso típico
main/master	Código productivo o versiones estables.
develop	Integración de funcionalidades antes de release.
feature/*	Una funcionalidad concreta. Nace de develop y vuelve a develop.
release/*	Estabilizar una versión: correcciones menores, documentación, versión.
hotfix/*	Corrección urgente de producción. Nace de main y vuelve a main/develop.

Ojo con GitFlow

No siempre conviene. Para un equipo de tres personas, un proyecto académico y entregas cortas, puede ser demasiado pesado. Podría usarse GitHub Flow: main estable + feature branches + pull request + tests antes del merge.

Estrategia simple recomendada para equipo chico

1. main siempre estable y ejecutable.
2. Cada integrante trabaja en feature/nombre-corto: feature/login, feature/api-users, feature/ui-users.
3. Antes de push: correr tests y revisar que compile.
4. Abrir pull request hacia main o develop, según la organización.
5. Otro integrante revisa el diff, comenta, pide cambios o aprueba.
6. Merge solo si pasan tests/checks y no hay conflictos.
7. Hacer pull frecuente para evitar divergencias grandes.

Qué revisar antes del merge

- Que el cambio responda al caso de uso o issue definido.
- Que no mezcle muchas cosas en el mismo PR: una feature o fix por PR.
- Que haya tests o al menos pruebas manuales documentadas.
- Que no haya secretos, contraseñas, tokens ni archivos generados innecesarios.
- Que respete arquitectura: controller delega, service orquesta, dominio valida reglas.
- Que no introduzca code smells obvios: God Class, duplicación, método largo.

```
# Flujo de trabajo ejemplo
git checkout main
git pull origin main
git checkout -b feature/api-users
# trabajar...
git add .
git commit -m "Agregar endpoints de usuarios"
git push -u origin feature/api-users
# abrir Pull Request en GitHub
# review + checks + merge
```

10. Monolito, monolito modular y microservicios

Monolito

Un monolito es una aplicación desplegada como una unidad. No significa automáticamente mal diseño. Puede estar muy bien organizado internamente en módulos y capas. El problema aparece cuando crece sin límites, se vuelve difícil de modificar, testear o desplegar, y cualquier cambio obliga a tocar o desplegar todo.

Microservicios

Una arquitectura de microservicios divide una aplicación en servicios independientes, usualmente alrededor de capacidades de negocio. Cada servicio puede tener su propia lógica, datos y ciclo de despliegue. Esto puede mejorar escalabilidad y autonomía, pero agrega complejidad distribuida.

AWS Prescriptive Guidance + Martin Fowler: microservicios, monolito y trade-offs.

Criterio	Monolito / monolito modular	Microservicios
Complejidad inicial	Menor. Más simple de desarrollar, probar y desplegar.	Mayor. Requiere red, observabilidad, CI/CD, seguridad entre servicios.
Escalabilidad	Se escala toda la aplicación.	Se pueden escalar partes específicas.
Despliegue	Una unidad de despliegue.	Servicios desplegables de forma independiente.
Datos	Base compartida o módulos internos.	Idealmente cada servicio controla sus datos.
Fallas	Una falla puede afectar todo si no se controla.	Puede aislar fallas, pero la comunicación remota también falla.
Equipo	Bueno para equipos chicos o dominio poco estable.	Útil con equipos más grandes y límites de dominio claros.
Testing	Más simple para integración local.	Requiere contrato, integración, resiliencia y observabilidad.

Cuándo NO conviene microservicios

- El equipo es chico y no tiene madurez de DevOps/CI/CD/monitoreo.
- El dominio todavía cambia mucho y no están claros los límites.
- La aplicación es pequeña o académica y se puede mantener con módulos internos.
- Se busca simplicidad para entregar rápido.
- No hay necesidad real de escalar partes por separado ni desplegarlas independientemente.

Cuándo sí puede tener sentido

- El monolito ya es demasiado grande y frena los cambios.

- Hay capacidades de negocio claras y relativamente independientes.
- Distintos módulos tienen ritmos de cambio o escalabilidad muy diferentes.
- Hay equipos separados que pueden hacerse responsables de servicios completos.
- Existe infraestructura para despliegue, monitoreo, logs, trazabilidad y manejo de fallos.

Estrategia de migración segura

8. Primero ordenar el monolito: separar capas, módulos y responsabilidades.
9. Identificar bounded contexts o capacidades de negocio: usuarios, pagos, reportes, notificaciones.
10. Extraer primero una capacidad simple y poco acoplada.
11. Definir contrato de API/eventos entre monolito y nuevo servicio.
12. Agregar tests de contrato e integración.
13. Migrar datos con cuidado: evitar compartir base de datos de forma indefinida.
14. Medir si la migración realmente mejora despliegue, escalabilidad o mantenibilidad.

Respuesta equilibrada

En un parcial suele sumar decir: 'No empezaría directamente con microservicios. Primero haría un monolito modular aplicando capas y SOLID. Si luego aparecen límites claros, escala diferencial o equipos independientes, migraría gradualmente.'

11. Seguridad web: JWT, middleware y guards

Este tema aparece mucho cuando se diseñan APIs. Aunque no esté desarrollado en las diapositivas base, conviene manejarlo para justificar autenticación y autorización.

JWT en pocas palabras

Un JWT es un token que transporta claims: afirmaciones sobre un sujeto. Tiene tres partes codificadas en Base64URL: header, payload y signature. Si está firmado, permite verificar integridad; pero firmado no significa cifrado. Por eso no se deben guardar contraseñas ni datos sensibles en el payload.

RFC 7519: JSON Web Token y consideraciones de privacidad.

Parte	Contenido	Ejemplo
Header	Tipo de token y algoritmo.	{ alg: 'HS256', typ: 'JWT' }
Payload	Claims: sub, email, roles, exp, iat.	{ sub: 10, role: 'admin', exp: ... }
Signature	Firma para verificar que no fue alterado.	HMACSHA256(header.payload, secret)

- Guardar identificadores y claims mínimos: userId/sub, roles, expiración.
- No guardar password, hash de password, DNI u otros datos sensibles.
- Usar HTTPS para transportar tokens.
- Definir expiración y renovación de tokens.
- Validar firma, expiración y permisos antes de acceder a recursos protegidos.

Middleware vs Guard en NestJS

Un middleware se ejecuta antes del route handler y puede leer request/response y llamar next(). Es útil para tareas generales: logging, parseo, agregar requestId o autenticación muy general. Un guard decide si una request puede llegar al handler. En NestJS los guards conocen el ExecutionContext y pueden leer metadata de la ruta, por ejemplo roles requeridos.

NestJS Docs: Middleware y Guards.

Aspecto	Middleware	Guard
Momento	Antes del handler.	Después de middleware y antes de pipes/interceptors/handler.

Conocimiento de ruta	Limitado: no sabe naturalmente qué handler exacto se ejecutará.	Tiene ExecutionContext y metadata del handler.
Uso ideal	Logging, requestId, parseos, autenticación simple.	Autorización, roles, permisos, acceso por endpoint.
JWT global	Puede validar token y adjuntar user si es general.	Mejor para decidir si esta ruta requiere auth y qué permisos.

Por qué no conviene validar todo JWT en middleware global

Porque no todas las rutas requieren las mismas reglas: algunas son públicas, otras requieren usuario y otras requieren rol admin. Un guard puede leer metadata del endpoint y aplicar autorización declarativa, evitando lógica rígida y dispersa.

12. Respuestas modelo basadas en el parcial de referencia

Estas respuestas no son para memorizar palabra por palabra, sino para copiar la lógica: diagnóstico + principio + propuesta + justificación.

Caso tipo 1: clase GestorUsuarios gigante

Enunciado: una clase GestorUsuarios registra usuarios, valida contraseñas, envía correos y audita accesos. Se quiere migrar a una arquitectura basada en microservicios y aplicar SOLID.

1) Problemas de diseño

- Viola SRP: tiene múltiples razones de cambio (registro, seguridad, correo, auditoría).
- Baja cohesión: junta tareas que no pertenecen al mismo objetivo interno.
- Alto acoplamiento: probablemente conoce base de datos, correo, validación y logs.
- God Class: concentra demasiada lógica y se vuelve difícil de entender.
- Dificulta testing: para probar registro se puede terminar dependiendo de correo real o auditoría real.
- Viola OCP: agregar un nuevo canal de correo o una nueva política de contraseña obliga a modificar la clase central.

2) Refactor aplicando SOLID

```
UserController
-> UserService.register()
    -> PasswordPolicy.validate()
    -> UserRepository.save()
    -> EmailSender.sendWelcome()
    -> AuditLogger.logUserCreated()
```

```
Interfaces:
UserRepository
EmailSender
AuditLogger
PasswordPolicy
```

- SRP: cada clase tiene una responsabilidad clara.
- DIP: UserService depende de interfaces, no de SMTP, SQL o proveedor concreto.
- OCP: para agregar SMS o WhatsApp, se crea otra implementación de NotificationSender.
- ISP: interfaces pequeñas: EmailSender solo envía email, AuditLogger solo audita.
- Testing: se mockean EmailSender y AuditLogger para probar UserService sin efectos reales.

3) Cómo facilita migrar a microservicios

Aplicar SOLID no convierte automáticamente el sistema en microservicios, pero deja límites más claros. Una vez separado el módulo de usuarios, notificaciones y auditoría, se puede decidir si alguno merece ser extraído como servicio independiente. Por ejemplo, AuditService podría pasar a un microservicio de auditoría si recibe eventos de muchas partes del sistema.

Respuesta corta posible

'Primero refactorizaría a un monolito modular. Si luego usuarios, notificaciones o auditoría requieren despliegue/escala independiente, extraería esos módulos gradualmente como microservicios, manteniendo contratos claros y tests.'

Caso tipo 2: equipo remoto y GitFlow

Enunciado: integrantes trabajan remoto desde distintos lugares y horarios. Preguntan cómo ayuda GitFlow, por qué usar ramas y PRs, y una estrategia antes del merge.

1) Cómo ayuda GitFlow

- Permite trabajo paralelo sin pisarse: cada feature vive en su rama.
- Separa código estable de código en desarrollo: main/develop.
- Ordena releases y hotfixes si el proyecto tiene versiones planificadas.
- Facilita trazabilidad: cada rama/PR se asocia a una tarea o issue.

2) Ramas y pull requests

- Una rama aísla el trabajo hasta que esté listo.
- El pull request permite revisar diff, discutir decisiones, detectar errores y ejecutar checks automáticos antes del merge.
- En remoto, el PR reemplaza parte de la conversación presencial: deja contexto, comentarios y aprobación documentada.

3) Estrategia antes del merge

15. Crear rama feature por tarea.
16. Commits pequeños y con mensajes claros.
17. Actualizar con main/develop antes de abrir PR para reducir conflictos.
18. Ejecutar tests y linter.
19. Abrir PR con descripción: qué cambia, cómo probar, capturas si aplica.
20. Pedir review de otro integrante.
21. Merge solo con aprobación, tests en verde y conflictos resueltos.

Matiz importante

Para un equipo muy chico, usar GitFlow completo puede ser innecesario. Se puede justificar una versión liviana: main estable + feature branches + pull requests + tests.

Caso tipo 3: API REST de usuarios

Enunciado: diseñar una API REST para administrar usuarios, consumible por web y móvil. Pedir endpoints, estructura de request/response y códigos HTTP.

Operación	Método y ruta	Código
Listar usuarios	GET /users? page=1&pageSize=20	200
Ver usuario	GET /users/{id}	200 / 404

Crear usuario	POST /users	201 / 400 / 409
Actualizar usuario	PUT /users/{id}	200 / 400 / 404
Actualizar parcialmente	PATCH /users/{id}	200 / 400 / 404
Eliminar usuario	DELETE /users/{id}	204 / 404
Asignar rol	POST /users/{id}/roles	201 / 404 / 409

```
POST /users
Content-Type: application/json
```

```
{
  "name": "Ana",
  "email": "ana@mail.com",
  "password": "abc12345"
}
```

```
HTTP/1.1 201 Created
{
  "id": 10,
  "name": "Ana",
  "email": "ana@mail.com"
}
```

Justificación: las rutas usan nombres de recursos, no verbos. Los métodos HTTP expresan la acción y los códigos informan el resultado. JSON permite que la misma API sea consumida por web y móvil. La lógica de negocio no debería estar en el controller sino en services/dominio.

Caso tipo 4: monolito vs microservicios para proyecto académico

Enunciado: alumnos comienzan un proyecto de software académico. Dudan entre arquitectura en capas o microservicios. Equipo de tres integrantes. Deben usar Git.

1) Decisión de arquitectura

Recomendaría comenzar con un monolito modular en capas, no con microservicios. Motivos: equipo chico, dominio probablemente cambiante, menor complejidad de despliegue y testing, y suficiente separación interna si se aplican capas y SOLID. Microservicios agregan costos de comunicación remota, despliegue, observabilidad y consistencia que no se justifican al inicio.

2) Organización en Git

- main estable.
- feature branches por tarea: feature/alumnos, feature/login, feature/importacion-excel.
- Pull requests cortos para revisión.
- No usar GitFlow completo salvo que haya releases formales; preferir flujo simple.
- README con cómo correr el proyecto, variables de entorno y convenciones.

3) Prioridades iniciales

- Definir casos de uso principales y modelo de dominio básico.
- Separar controller, service, domain/repository desde el comienzo.
- Definir endpoints REST y contratos JSON.
- Configurar Git, ramas, PRs y .gitignore.
- Agregar tests unitarios de reglas clave.
- Configurar manejo global de errores y validaciones.
- Evitar sobreingeniería: aplicar KISS/YAGNI.

13. Preguntas de práctica

Pregunta 1

Una clase ReportManager genera reportes, los guarda en PDF, los sube a Drive y envía emails. ¿Qué problemas detectás y cómo refactorizarías?

Respuesta esperada

SRP, God Class, alto acoplamiento. Separar ReportGenerator, PdfExporter, StorageService, EmailSender. Usar interfaces para almacenamiento y correo.

Pregunta 2

Diseña endpoints para cursos y alumnos donde un curso tiene muchos alumnos.

Respuesta esperada

GET /courses, GET /courses/{id}, POST /courses, GET /courses/{id}/students, POST /courses/{id}/students, DELETE /courses/{id}/students/{studentId}.

Pregunta 3

¿Por qué no conviene que el Controller consulte directamente la base?

Respuesta esperada

Porque mezcla transporte HTTP con persistencia y negocio. Aumenta acoplamiento, dificulta testing y rompe separación de capas.

Pregunta 4

¿Cuándo usarías 400, 401, 403, 404 y 409?

Respuesta esperada

400 datos inválidos; 401 no autenticado; 403 sin permisos; 404 recurso inexistente; 409 conflicto como email duplicado.

Pregunta 5

¿Qué diferencia hay entre condición de prueba y caso de prueba?

Respuesta esperada

Condición es la situación a verificar; caso son datos concretos para ejecutar esa condición.

Pregunta 6

¿Qué beneficio tiene un PR en equipos remotos?

Respuesta esperada

Permite revisar cambios, discutir el diff, ejecutar checks y dejar trazabilidad antes de mergear.

Pregunta 7

¿Por qué microservicios no siempre son mejor que monolito?

Respuesta esperada

Agregan complejidad distribuida: red, fallos parciales, observabilidad, despliegue, consistencia eventual. Si el sistema es chico, conviene monolito modular.

14. Machete final antes del parcial

Palabras clave que conviene usar

Tema	Palabras clave
Diseño	cohesión, acoplamiento, mantenibilidad, extensibilidad, robustez, simplicidad
SOLID	SRP, OCP, LSP, ISP, DIP, interfaces, abstracciones, polimorfismo
Capas	controller, service, dominio, repository, DTO, validación, error handler
REST	recurso, URI, método HTTP, JSON, status code, idempotencia, paginación
Git	branch, commit, pull request, merge, review, conflicto, main estable
Testing	condición de prueba, caso de prueba, resultado esperado, mock, unit test
Microservicios	bounded context, despliegue independiente, escalabilidad, observabilidad, complejidad distribuida

Plantillas de respuesta

Diagnóstico:

'El problema principal es que ... Esto genera ...'

Principio:

'Se relaciona con ... porque ...'

Propuesta:

'Lo refactorizaría separando ... e introduciendo ...'

Justificación:

'Con esto se mejora ... ya que ... El costo/trade-off es ... pero se justifica porque ...'

Checklist para diseñar una API REST

- ¿Identifiqué recursos y no acciones?
- ¿Usé métodos HTTP correctos?
- ¿Definí JSON de entrada y salida?
- ¿No devuelvo información sensible?
- ¿Definí códigos 200/201/204/400/401/403/404/409/500?
- ¿Agregué paginación en listados grandes?
- ¿Separé controller de service/dominio?

Checklist para justificar arquitectura

- ¿El equipo y el proyecto justifican la complejidad elegida?
- ¿La solución mejora mantenibilidad o solo agrega capas por agregar?
- ¿Los límites de responsabilidad son claros?
- ¿Puedo testear reglas sin levantar toda la infraestructura?
- ¿Hay una estrategia de Git coherente con el tamaño del equipo?
- ¿Estoy aplicando KISS/YAGNI o sobrediseñando?

15. Fuentes consultadas

Fuentes de cátedra:

- [DDS UTN FRVM] Clase Semana - 02.pdf: arquitectura cliente-servidor, HTTP, arquitectura web, API, API REST y recursos web.
- [DDS UTN FRVM] Clase Semana - 03-1.pdf: orquestación de casos de uso, capa de dominio, repositorios, services, controladores, validaciones y errores.
- [DDS UTN FRVM] Clase Semana - 05-1-1.pdf: cualidades de diseño, SOLID, code smells, testing, Jest y mocking.

Fuentes externas usadas para complementar:

- MDN Web Docs - HTTP request methods: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>
- MDN Web Docs - Códigos de estado HTTP: <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status>
- Git Book - Basic Branching and Merging: <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
- GitHub Docs - About pull requests: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- GitHub Docs - Reviewing proposed changes in a pull request: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/reviewing-proposed-changes-in-a-pull-request>
- AWS Prescriptive Guidance - Decomposing monoliths into microservices: <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-decomposing-monoliths/welcome.html>
- Martin Fowler - Microservices y Microservice Premium: <https://martinfowler.com/articles/microservices.html> ; <https://martinfowler.com/bliki/MicroservicePremium.html>
- Jest Docs - Mock functions/manual mocks: <https://jestjs.io/docs/mock-functions> ; <https://jestjs.io/docs/manual-mocks>
- RFC Editor - RFC 7519 JSON Web Token: <https://www.rfc-editor.org/info/rfc7519>
- NestJS Docs - Middleware y Guards: <https://docs.nestjs.com/middleware> ; <https://docs.nestjs.com/guards>

Fin del resumen